ELEC 392: Principles of Design and Development

Final Report for the Development of a Safety-First Autonomous Taxi System to Improve Transportation in Quackston

Team ID: Blekinge 12

Firm Name: Rust-eze

Firm Partners:

Ben Malvern – 20424655 - 23DJK

Clarke Needles- 20424709 – 23GPB

Filip Radenovic - 20417483 – 22LJ43

Jimmy Moutafis-Tymcio – 20420446 – 23PVVB

Prepared For: Mohammad Ramezani Soukhtekouhi, Town of Quackston

Submitted April 6, 2026

Statement of Originality

We ensure that this report upholds the academic integrity principles as defined by Queen's University. We declare that this report is our own work and that any tools or ideas that have been used in formulation of this report have been properly cited or acknowledged. We adhere to the Queen's University principles of academic integrity such as honesty, trust, fairness, respect, and responsibility and understand that these principles apply to all aspects of our work. There is no plagiarism, unauthorized assistance, or any form of academic misconduct. Generative AI was used in the making of this report solely to assist with formatting, wording, grammar, and report structure. All ideas and designs are entirely the team's original work. AI-generated text was reviewed and edited by the team members to ensure accuracy, and all information used has been properly cited and acknowledged. The team takes full responsibility for the originality of the final report.

By signing below, the team members affirm the accuracy of the above statement.

Clarke Needles Signature: _____C.N_____ Date: _____April 6, 2026_____

Ben Malvern Signature: _____B.M_____ Date: _____ April 6, 2026_____

Jimmy Moutafis-Tymcio Signature: _____Jimmy M-T_____ Date: _____April 6, 2026_____

Filip Radenovic Signature: _____F.R_____ Date: _____April 6, 2026_____

Work Breakdown

Table 1: Work Breakdown Table

Team Member	Sections Written and Additional Contributions
Ben Malvern	4.1 Engineering Design Process 4.2 System Architecture 5.2 Integration Strategy
Clarke Needles	Implementation and Integration System Architecture Diagrams
Filip Radenovic	Introduction Executive Summary Ethical, Social, and Environmental Considerations Appendix A Appendix C
Jimmy Moutafis-Tymcio	Design Criteria and Specifications Design Methodology and System Architecture (4.4-4.7) Verification, Validation, and Performance Evaluation Limitations, Risks and Recommendations Conclusion

Table of Contents

List of Figures

List of Tables

Executive Summary

The Town of Quackston requires a modern transportation solution to handle its rising population and the unsustainability of passive transit. The team developed a safety-first autonomous taxi system to navigate this urban environment while following strict traffic regulations. This project utilized the PiCar-X platform to create a vehicle capable of lane following, obstacle avoidance, and passenger transport through the city's Vehicle Positioning and Fare System (VPFS).

The system architecture relies on a modular ROS 2 Humble framework running on Ubuntu Server. This design separates the autonomy stack into distinct nodes for perception, behavior management, navigation, and hardware control. A primary technical feature is the use of a semantic segmentation model accelerated by a Google Coral Edge TPU. This model classifies pixels into background, green lane markings, and yellow stop lines with a validation accuracy of 98.9%. By choosing vision-based perception over traditional grayscale sensors, the vehicle identifies road features further ahead and maintains higher operational speeds.

Safety serves as the fundamental design pillar. The team implemented a Cabin and Restraint Equipment (CARE) system to keep duck passengers secure during movement. The vehicle includes functioning brake lights, turn signals, and an audio-visual alert system to communicate its intentions to others. A dedicated ultrasonic sensor provides a fail-safe layer by triggering an immediate emergency stop if it detects a pedestrian or obstacle.

Validation trials confirm the effectiveness of the integrated design. The vehicle successfully completed 100% of its straight line and left turn trials. It also achieved a 100% success rate across 20 stopping trials. In full system integration tests, the taxi successfully completed 50% of its autonomous fare runs. Qualitative observations highlight that the vehicle possesses superior speed and maneuverability compared to other systems in the same environment.

The final competition run did not reflect these validated capabilities due to a procedural software deployment error. A version control issue caused the local codebase to revert to an older iteration before the event. While this prevented the best performing code from running during the competition, controlled testing results prove the underlying hardware and logic are sound.

The project demonstrates that a vision based modular architecture provides a reliable foundation for autonomous urban transit. Future work should focus on integrating grayscale sensors as a fallback layer and expanding training data to handle inconsistent lighting. The team recommends earlier end to end testing and stricter version control protocols to ensure reliable deployment in competitive or real-world settings.

1. Introduction

1.1 Background Information

Autonomous driving is a rapidly advancing field of robotics and artificial intelligence, with applications ranging from large-scale commercial transportation to small-scale educational platforms. The core challenge of autonomous vehicle design lies in integrating perception, localization, planning, and control systems into a single cohesive pipeline that can operate reliably in dynamic, real-world environments.

Engineers typically separate driving tasks into distinct stages, where modular pipelines handle perception, localization, planning, and control independently. This structure simplifies debugging [20][21]. End-to-end learning offers an alternative, where a single deep neural network maps raw sensor data to control commands [21][34]. Small robots use single-stage detectors for speed; YOLO models detect pedestrians and signs despite occlusion [37][23][24][36]. Classical techniques identify lane markings using the Canny edge detector and Hough Transform [25][35]. Graph-based algorithms such as Dijkstra and A* find collision-free paths between nodes [20], while PID controllers and geometric controllers such as Pure Pursuit and Stanley methods maintain speed and correct steering [27][28].

Complex inference models require specialized hardware. The Google Coral Edge TPU performs 4 trillion operations per second on 2 watts [29], enabling real-time inference through INT8 quantization [30]. Simulators such as Gazebo allow developers to use digital twins for training, reducing the performance gap caused by physics differences [26][31][37].

1.2 Problem Statement and Motivation

Due to the rising population of Quackston, the reliance on the ducks' traditional method of transit, passively floating to destinations, has become unsustainable. This has prompted the town to modernize its infrastructure with an autonomous taxi network. The team aimed to design, build, and test a next-generation autonomous vehicle capable of navigating Quackston's urban landscape while finding and dropping passengers off through the city's Vehicle Positioning and Fare System (VPFS). The core objective was to engineer a system that balances operational efficiency with the strict ethical mandates of the Quackston Road Safety Act, ensuring compliance with all traffic laws, staying within lanes, obeying traffic signs, driving through roundabouts, both one-way and two-way streets, while prioritizing the absolute safety of rubber ducky passengers and pedestrians above all else [38].

1.3 Structure of the Report

This report is organized as follows: Section 2 defines the design criteria and specifications, including system requirements, objectives, and scope. Section 3 addresses the ethical, social, and environmental considerations guiding the design. Section 4 describes the design methodology and system architecture. Section 5 covers implementation and integration of hardware and software components. Section 6 presents verification, validation, and performance evaluation results. Section 7 discusses limitations, risks, and recommendations for future work. Section 8 provides the conclusions. References and appendices follow.

2. Design Criteria and Specifications

2.1 System Requirements and Constraints

The design of the autonomous taxi was constrained by physical, safety, operational, and ethical requirements set by the project specifications. These constraints influenced both the hardware implementation and the overall system design.

A key physical constraint was the vehicle size limit of 300 mm in height, 170 mm in width, and 300 mm in length. Although there was no weight limit, the vehicle still needed to remain stable during operation. The base car could not be permanently modified. Components could be added, removed, or adjusted as needed, provided the vehicle could be reassembled into its original configuration. As a result, all hardware integrations had to be designed in a way that was reversible, which constrained mounting methods and encouraged a compact and organized build.

Safety requirements were also considered. The vehicle needed an emergency stop system that could immediately halt motion, override autonomous operation, and be accessible. It also had to include the CARE restraint system to ensure ducks remained secure without tipping, sliding, or being ejected. In addition, wiring, batteries, and all electrical components had to be properly secured with no exposed components. Since unsafe operation could result in a stopped run or disqualification, safety had to take priority throughout the design.

The project also required rear brake lights, rear left and right turn signals, and an audio-visual alert system consisting of both a horn and a visible warning output. These systems had to function both autonomously and manually. Driving behaviour was similarly constrained, as the vehicle had to operate fully autonomously during runs, drive smoothly, stay in its lane, stop completely at markers, and avoid all contact. Due to the fact that any contact counted as a collision and rolling stops were still penalized, the control system had to prioritize precision and reliability

Another constraint was compatibility with Quackston's VPFS tracking system. A printed AprilTag had to be mounted flat, level, visible, and unobstructed, which further limited the arrangement of hardware on the vehicle.

Finally, the design was shaped by ethical and strategic constraints. Since performance was evaluated through both technical success and its effect on cash flow and reputation, safety was treated as the first priority. The team's goal was to first achieve safe, stable, and reliable autonomous operation, and only after that to improve performance through refinements such as smoother control and increased speed. This approach reflected the need to balance competitiveness with safety and trust.

2.2 Project Objectives

The primary objective of this project was to design, build, and test an autonomous taxi system for Quackston that prioritized passenger safety while operating reliably and in compliance with competition requirements. The vehicle was intended to navigate to destinations throughout the town, follow traffic regulations, maintain lane control, transport ducks securely, and avoid collisions with obstacles or pedestrians. Additional objectives included interfacing with the VPFS, detecting relevant road features and markings, and completing fares with no manual intervention. Success was measured by the vehicle's ability to perform these tasks safely, consistently, and efficiently within the project constraints and competition environment.

2.3 Scope

This project focused on the design, construction, and testing of a fully autonomous taxi system for operation in Quackston. The scope of the project was limited to the course competition environment, which consisted of a small urban-style road network with lanes, lines, intersections, traffic signs, obstacles, and duck passengers. Within this setting, the vehicle was intended to operate as an integrated hardware-software system capable of autonomous navigation while complying with the functional and safety requirements of the competition.

The project scope included autonomous navigation between fare locations, lane following, stopping behaviour, obstacle avoidance, passenger restraint, emergency stop functionality, signalling, and compatibility with VPFS. It also included the integration of onboard sensors, embedded computation, and control software needed to support reliable autonomous operation on the PiCar-X platform. For line following, the perception scope included the development and deployment of a semantic segmentation TensorFlow Lite model to identify the relevant features needed for steering and path tracking.

The project scope did not include broader claims of deployment beyond the controlled course environment. It was not intended to solve full-scale real-world autonomous driving challenges, nor to implement unnecessarily high-overhead methods that exceeded the needs of the competition. Instead, the scope was to produce a safe, functional, and competition-compliant prototype that balanced technical performance with safety, reliability, and overall competitive success.

2.4 Success Criteria — Functional Requirements

Table 2: Functional Requirements Table

#	Requirement	Pass	Fail
1	The vehicle shall be able to accurately follow a line and stay in its lane during operation.	X	
2	The vehicle shall detect stop conditions, such as stop or yield markings, and obstacles, and respond appropriately.	X	
3	The vehicle shall include an emergency stop system that immediately overrides vehicle motion.	X	
4	The vehicle shall interface with the VPFS and be able to pick up and drop off fares.	X	
5	The vehicle shall securely restrain a duck passenger using the CARE system.	X	
6	The vehicle shall provide functioning rear brake lights, left and right turn signals, and an audio-visual alert system.	X	
7	The vehicle shall integrate sensing, computation, signalling, and actuation into one functioning autonomous vehicle.	X	
8	The vehicle shall contain and secure all wiring, electrical connections, and mounted components so that no parts are loose or exposed during operation.	X	

2.5 Success Criteria — Performance Requirements

Table 3: Performance Requirements Table

	Criterion	Not at All	Somewhat	Mostly	Completely
1	The vehicle should avoid unsafe lane departures, excessive oscillation, or unstable steering corrections.				X
2	The vehicle should complete turns and steering corrections smoothly without excessive overshoot.			X	
3	The vehicle should be able to claim a fare, determine an optimal path to the pickup location, navigate there abiding by all traffic rules and stopping until the fare is picked up. Once picked up, it should determine a path to the dropoff point and autonomously navigate there abiding by all traffic rules, stopping at the dropoff point until the fare has been completed. Upon completion, it should select a new fare and repeat this process.			X	

3. Ethical, Social, and Environmental Considerations

3.1 Ethical Framework and Design Philosophy

The team follows a clear three-step ladder for system priorities. The first step is to establish baseline functionality. The second is to ensure operational safety. Only after those are achieved does the team optimize for fare performance. This hierarchy is grounded in Mill's utilitarianism: the objective is to pursue what provides the greatest benefit to the most duckizens. A safe, predictable vehicle serves the town more effectively than a fast vehicle that crashes. As a result, function and rule-following are treated as the primary design goals.

The design also draws on duty ethics. The team is relied upon to act responsibly, so safety is treated as a non-negotiable obligation rather than a preference. The Quackston Road Safety Act is followed because it represents a duty, not merely because it is convenient. The Challenger disaster serves as a cautionary example of what can happen when teams override safety data under deadline pressure or outside influence. Safety data is trusted over deadlines and external pressures. The design also strives for balance. In Aristotle's terms, this represents the golden mean: the vehicle is not so slow as to be useless, nor so aggressive as to be reckless.

3.2 Safety and Fairness

Safety is the primary design priority. A Cabin and Restraint Equipment (CARE) system was built using duct tape and cardboard. Though simple in construction, it is effective: it keeps passengers secure

during sharp turns and sudden stops. Vehicle intentions are made explicit through signalling. Red lights indicate braking; amber lights indicate turning. Pedestrians and other road users can always anticipate what the vehicle will do next. The vehicle is also capable of stopping nearly instantly, as the emergency stop overrides the entire system and cuts motion immediately. If something goes wrong, the operator remains in control.

It is well established that AI models can struggle to accurately identify individuals with darker skin tones. The team considered this bias during the design process. This is not currently a safety concern for this vehicle, as an ultrasonic sensor is used for pedestrian detection rather than a vision-based classifier. The sensor triggers an immediate stop regardless of the appearance of the person detected. Should the system be extended to more complex roads or higher speeds in the future, the perception model will be trained on a wide and diverse dataset to keep it as unbiased as possible. The team also 3D printed the vehicle shell to provide a professional and sturdy exterior. If the vehicle were to operate at higher speeds that pose greater risk to passengers, different materials with crumple zones would be used.

3.3 Societal Implications

Quackston is a rapidly growing town. Passive floating is no longer a sustainable method of transit given the increasing population. The autonomous taxi network is intended to help the town modernize its infrastructure in a practical and scalable way. Public adoption of self-driving vehicles depends heavily on trust in the technology. This trust is built by treating safety as the most important design goal throughout the entire project.

The team also considered the trolley problem, a well-known ethical dilemma in which an autonomous system must make a moral choice during an unavoidable collision scenario. A specific programmatic solution to this dilemma was not implemented in the current vehicle. Because the vehicle operates at very low speeds, a situation where a serious collision is unavoidable after detection is not considered a realistic concern at this stage. The team also values informed consent. Pedestrians present in the operating environment have not agreed to participate as test subjects. They are protected by validating all software in simulation before moving to the physical track.

3.4 Sustainability and Environmental Impact

The project was designed with environmental responsibility in mind. The hardware is intentionally efficient: a Google Coral Edge TPU is used, which performs four trillion operations per second while drawing only two watts of power. This low power draw supports the town's goal of a more energy-efficient transportation infrastructure and promotes a greener future. Materials were selected to minimize waste. Recycled cardboard and 3D printed components were used for the vehicle structure, along with common fasteners and duct tape for the interior. These choices help avoid unnecessary material waste. 3D printing allows parts to be produced only as needed, avoiding overproduction. By choosing sustainable materials and energy-efficient hardware throughout the design process, the overall carbon footprint of the project is reduced

4. Design Methodology and System Architecture

4.1 Engineering Design Process

The engineering design process was heavily focused on early planning, followed by iterative testing with refinement. Because of the complexity of the system, the project was broken down into multiple sub-systems. This included low-level vehicle motion, lane following, object detection, localization, navigation, and behavior management. Early in the process, the team created a high-level functional decomposition of the project that separated it into major problem areas.

This crucial step in the design process helped the team identify the largest problem areas and assign responsibility to those challenges. The team then worked to create a ROS node map to determine how information could flow efficiently and effectively between all these subsystems. Each of these nodes could be assigned to team members to work on the specific implementation, without affecting the functionality of other subsystems.

This planning stage was especially important because the system was very safety-sensitive and was physically embodied. Failures in perception, control, or communication could result in unsafe vehicle behavior. For this reason, the team wanted to carefully plan all higher-level logic, to ensure safety was paramount, and that all safety features could terminate operation if needed. For example, priority had to be given to emergency stop when a fault occurred such as a pedestrian being detected in front of the vehicle or losing the lane line.

Once the overall architecture was established, development became much more iterative. For subsystems such as camera streaming, semantic segmentation, lane following, turn execution, and VPFS integration were developed separately then tested on the physical vehicle in stages. Based on the outcome of each of the tests, ROS topics were refined, gains were tuned, and thresholds were adjusted.

Overall, the project utilized careful initial planning of the system architecture with incremental implementation and repeated physical testing, which made the final system more modular, easier to debug, and better suited to the competition environment.

The system design visualization is in Appendix C as figures 1 and 2.

4.2 System Architecture

The final system was designed as a modular ROS 2 software and hardware architecture built for the PiCar-X platform. The onboard autonomy stack ran on a Raspberry Pi using Ubuntu Server and ROS 2 Humble. Rather than treating the system as a single large program, the system was divided into major functional subsystems. These subsystems were responsible for behaviour management, perception and line following, hardware interfacing, and positioning and fare handling. This can be seen as figure 3 in Appendix C.

4.2.1 Behaviour Management

The behavior management node acted as the high-level decision maker. It ran at a fixed polling rate and operated as a finite state machine (FSM). It served as the main controller for the vehicle's current mode of operation and coordinated when the robot should wait, drive, stop, or transition between fares.

Based on events such as a line detection, stop-line detection, obstacle detection, fare updates, and route progress, the FSM would decide which mode of operation it needed to be in. These states included behaviours such as wait for fare, enter line following mode, handling stop/emergency conditions, executing turns, and completing pickup / drop-off actions. By centralizing these decisions, the system could separate high level decisions from low level drivers and logic.

4.2.2 Positioning, Mapping and Fare System

The positioning and fare subsystem handled the vehicles external task information and route context. This subsystem interfaced the VPFS system to receive task data and vehicle pose data. The critical output of this node was a queue of the next required turns the robot had to make to get to its destination.

To choose a fare efficiently, each fare was assigned a score which was calculated by taking the payout value divided by the total distance required to grab and complete the fare. Once the highest efficiency fare was received, the mapping system found the required node path using Dijkstra's algorithm on a distance weighted map to find the shortest path. Then, starting with the robot's current location, it would iterate through each node and figure out which direction the robot needed to turn from each node to get to the next node. This information was then pushed to the FIFO queue.

As the robot progressed through each node, the behaviour manager would signal when a turn was complete and pop the direction form the queue. If the robot ever lost its location on the path, it was recalculated and refreshed.

4.2.3 Perception & Line Following

The perception and line following subsystem were responsible for interpreting the road using a semantic segmentation model and generating immediate instructions to publish as ROS velocity commands to stay on the line using a PD controller or execute turns / stops.

The perception subsystem made calculated interpretations of what it was seeing and would execute movement based on input from the VPFS turn queue, and other algorithms / models discussed later in section 4.4. It was given exclusive control to the motors but was only allowed to publish velocity commands when the behaviour manager raised a flag signaling that it was allowed to.

When executing turns at an intersection, the line following subsystem entered tank steer mode, which allowed it to tell hardware that it wants to turn in place until it finds the desired line. In addition to this, to ensure the robot turned on the correct intersection, it could detect when there was a 2-lane road crossing over its current lane. This allowed it to drive past the first lane intersection on left turns, to ensure it always stayed in the correct lane, and didn't count as multiple turns being executed.

4.2.4 Hardware Interfacing

At the hardware interface layer was responsible for converting ROS-level control into physical actions on the PiCar-X platform. At the hardware level, ROS velocity commanded were interpreted as normalized linear and angular motion requests, which were converted into motor speed and steering servo commands for the vehicle.

This layer also handles lower-level actuator-related functions such as tank turning, turn signals, brake lights, and other obstacle related safety responses. Keeping all of these responsibilities in dedicated hardware nodes allowed the higher-level autonomy system to remain independent of the exact hardware and electrical connections.

4.3 Algorithms and Models

The system used a semantic segmentation model for lane perception and several supporting algorithms for control, navigation, and safety. The perception pipeline processed camera frames using a custom TensorFlow Lite semantic segmentation model accelerated on the Google Coral Edge TPU. This model classified each pixel into one of three classes: background, green lane markings, and yellow stop lines. By producing a pixel-level mask instead of only object detections, the model allowed the vehicle to identify the drivable lane region and detect stop markings directly from the camera image.

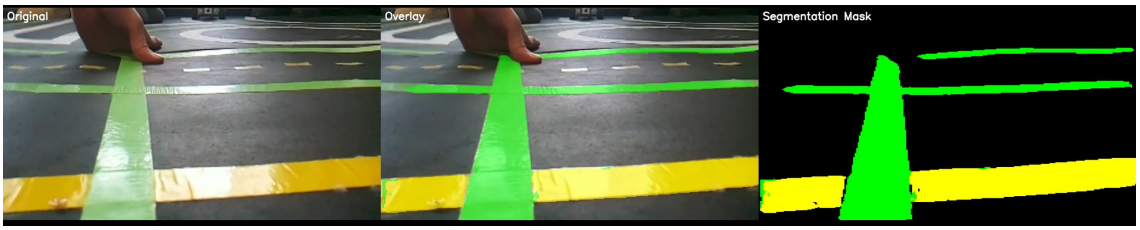

Figure 1: Segmentation Mask Showing the Mask Compared to Raw Camera Frame

After inference, the segmentation mask was cleaned before control decisions were made. Morphological opening and closing were used to remove small noise regions, and connected component analysis was used to isolate the most likely lane. The selected segment was chosen using factors such as size, closeness to the image center, and how far it extended toward the bottom of the frame. This ensured that the controller followed one consistent feature rather than reacting to multiple detections.

For steering control, the vehicle used a PD controller. The proportional term corrected the current lateral error between the detected lane center and the image center, while the derivative term reduced oscillation by responding to how quickly that error was changing. Additional smoothing and rate limiting were applied so that steering changes remained stable and the vehicle did not react too aggressively to noisy frames. This allowed the car to track the lane with smoothly.

The decision-making logic was implemented as a finite state machine. Different states handled behaviours such as normal line following, preparing for turns, executing tank turns, stopping at stop lines, and recovering when the lane was lost. A state-based design was used so the vehicle could respond differently depending on the situation.

For route planning, the system used Dijkstra's algorithm on a directed graph representation of the Quackston road network. Intersections were represented as nodes and valid road segments as weighted edges. Dijkstra's algorithm was used to compute the shortest valid path from the current location to the target fare location. After a path was generated, consecutive node triples were analyzed using vector cross products and dot products to classify each maneuver as a left turn, right turn, or straight movement. This turn queue was then passed to the autonomy system for execution at upcoming intersections.

Obstacle handling used ultrasonic sensing as a fail-safe safety layer. Distance readings were filtered, and repeated detections within a threshold distance triggered an obstacle flag. This allowed the higher-level behaviour logic to pause driving when an obstacle was present, providing a simple and reliable low-speed collision prevention mechanism.

Together, these models and algorithms formed a modular autonomy pipeline in which semantic segmentation handled visual lane perception, PD control handled steering, the finite state machine handled behaviour transitions, Dijkstra's algorithm handled route planning, and ultrasonic sensing supported safety-critical stopping.

4.4 Training and Data Management

Training data for the semantic segmentation model was collected directly from the vehicle's onboard camera in the Quackston environment, both on the setup mats and final mat, and then labeled in Roboflow to generate pixel-wise masks for the relevant classes. To improve robustness, the dataset was augmented with simulated motion blur, brightness variation, shear, exposure changes, and image noise so that the model would better generalize to changing lighting and motion conditions. The exported dataset was then used in Google Colab, where images and masks were resized, normalized, and divided into training, validation, and test sets. A lightweight MobileNetV2-based semantic segmentation model

was trained in two stages: an initial phase with the backbone frozen so that the decoder could learn the segmentation task, followed by a fine-tuning phase with part of the backbone unfrozen at a lower learning rate. Model checkpointing, learning-rate reduction, and early stopping were used to improve generalization and retain the best performing weights. During training, performance was monitored using loss and pixel-wise accuracy on the validation set, with the best validation results reaching approximately 98.9% accuracy and a validation loss of 0.0323 before training was stopped and the model was restored to the best epoch. The final trained model was then converted and quantized to TensorFlow Lite for deployment on the Coral TPU.

4.5 Alternatives Considered and Trade-offs

Several design alternatives were considered throughout the project, particularly in the areas of lane perception and software architecture. These decisions were guided by the need to produce a system that was both functional and robust within the Quackston environment.

One important trade-off involved the choice of sensor for line following. An alternative approach was to use the grayscale sensor mounted on the vehicle. While this method offered a simple and lightweight way to detect a line directly beneath the car, it provided only very local information and could not observe the road ahead. In contrast, the camera-based approach allowed the vehicle to capture a larger portion of the environment in front of it, including the lane shape, upcoming turns, and stop markings. This allowed our vehicle to perform things that none other could, such as stopping a safe distance behind stop signs, and driving and turning at speeds significantly faster than the grayscale allowed. For this reason, the camera was considered the more robust option for autonomous driving, since it provided richer visual context and allowed control decisions to be made earlier.

Another major design choice involved the choice of operating system. Instead of using a more standard Raspberry Pi OS, the project was built on Ubuntu with ROS2. This introduced additional setup and integration complexity, but it provided important advantages in modularity, communication, and system organization. ROS2 allowed the project to be divided into separate nodes for perception, behaviour, navigation, localization, and hardware control, making the full system easier to structure and debug. This choice was also motivated by the team's desire to gain practical experience working with ROS, since it is widely used in robotics and autonomous systems development. As a result, the team accepted the added complexity of ROS and Ubuntu in exchange for improved system modularity and valuable experience with a professional robotics software framework.

4.6 Technologies and Tools

The system was built using a combination of embedded hardware, sensors, robotics software, and machine learning tools. The PiCar-X provided the chassis, steering servo, motors, and base vehicle structure. A Raspberry Pi was used as the onboard computer for running the autonomy stack, while a Raspberry Pi camera provided visual input for line following and stop-line detection. A Google Coral Edge TPU was used to accelerate the semantic segmentation model during real-time operation. Additional hardware included an ultrasonic sensor for obstacle detection and LEDs for brake lights and turn signals.

On the software side, Ubuntu and ROS 2 were used to develop and run the autonomous vehicle system in a modular node-based architecture. Python and C++ were used for implementing perception, control, behaviour, localization, and hardware interface nodes. The computer vision model was developed using TensorFlow and converted to TensorFlow Lite for deployment on the embedded platform, while OpenCV was used for image preprocessing and frame handling. Model training was carried out in

Google Colab, and Roboflow was used for dataset labeling, augmentation, and export. GitHub was used for version control and project organization.

5. Implementation and Integration

5.1 System Components

The system was designed to have a modular architecture with thoughtfully connected hardware and software components by utilizing the ROS2 framework. To support consistent development and deployment, Docker containers were used to provide a standardized environment across all team members. This enabled parallel development and testing of individual subsystems so the components could be refined independently before combining them into a unified system. Compared to the original system architecture in Figure 3, the final implementation is slightly different. The physical subcomponents have all been incorporated, but the structure itself may be slightly different. Nonetheless, the functionality of the system is the exact same. The final system architecture is shown below.

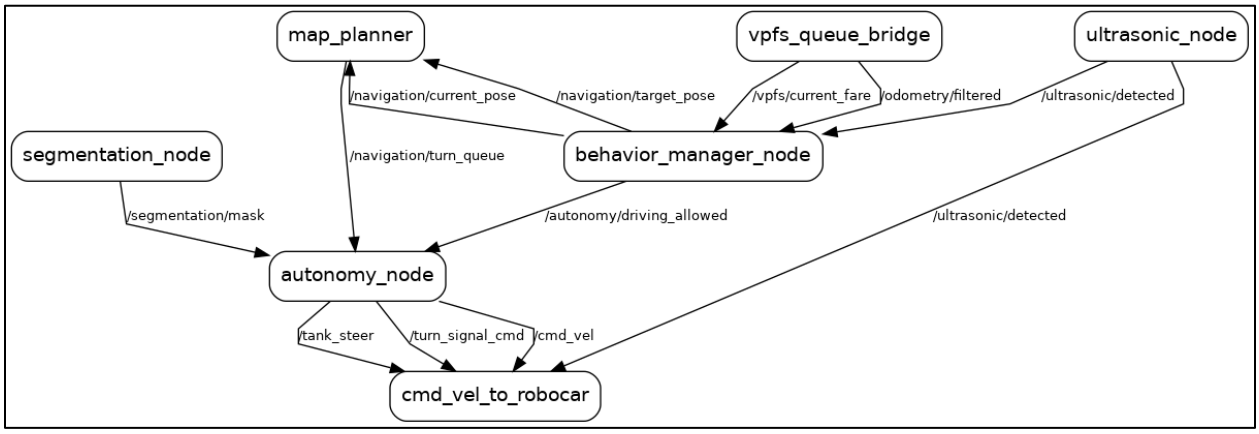

Figure 2: Final ROS2 System Architecture

5.1.1 Hardware Subsystem

The hardware subsystem provides the interface between ROS2 software, and the PiCar-X hardware controls. This subsystem is all within the *hardware_pkg* and is responsible obtaining sensor data while also controlling the steering and motors. This package consists of the *cmd_vel_to_robocar* and *ultrasonic* nodes, both Python in python for seamless implementation with Robot Hat drivers, also written in Python.

In the *cmd_vel_to_robocar* node, high-level velocity commands received through the topic */cmd_vel* and are translated into low-level motor signals for the DC motors and steering servo via the Robot Hat driver modules. This node controls hardware for the brake lights and turning signals by looking at the */cmd_vel* and */turn_signal_cmd* topics respectively. When turning, the *tank_steer* topic is utilized. The hazard lights are controlled by the detection of an object from the */ultrasonic/detected* topic. The ultrasonic node controls all the sensor data from the ultrasonic sensor. When an object is within the threshold, it notifies the listeners through the */ultrasonic/detected* topic.

This subsystem is fundamental to how the system works. This package abstracts the hardware details away from the rest of the system, making it very easy to integrate with higher level subsystems.

5.1.2 Perception Subsystem

The perception subsystem (*perception* package*)* is essentially the autonomy node. It is responsible for interpreting the robot's surroundings, and in this case, mostly through computer vision. The *segmentation_node* is used to detect and follow a path of green tape. It will output a segmentation mask to the *autonomy_node* through the */segmentation/mask* topic.

Using this input, the *autonomy_node* performs lane_following and low-level driving logic, through an PID controller integrated into the node. The outputs are velocity commands (*/cmd_vel*), steering mode signals (*/tank_steer*), and turn signal commands (*/turn_signal_cmd*). This node is the main control of the motors along with the PID. This takes some of the load off the behavior manager and makes the motor control easier to understand. This package was developed in Python because of the computer vision libraries that already exist in Python.

5.1.3 Localization Subsystem (VPFS integration)

The localization subsystem (*localization_pkg*) is implemented through the *vpfs_queue_bridge*, which interfaces with the external VPFS. This node gets the updated position of the car and active fare from the VPFS server and then publishes the car's position to */odometry/filtered* and the fare to */vpfs/current_fare*.

By giving a global position reference and task information, this subsystem enables coordinate navigation with the mapping component for proper task execution within the environment. This node was coded in Python because the ease of implementation with the VPFS Python libraries.

5.1.4 Navigation Subsystem

The navigation subsystem (*navigation_pkg*) is implemented through the *map_planner* node, which is responsible for generating paths between the car's current position and its target destination. This node subscribes to */navigation/current_pose* and */navigation/target_pose*, which are provided by the behavior management subsystem. Using a predefined map structure, it computes the optimal route and publishes a sequence of turn instructions to */navigation/turn_queue*.

By providing a structured set of navigation commands, this subsystem enables the robot to follow a planned route and execute directional decisions at intersections. This node was coded in C++ to ensure efficient path computation and timely updates in response to changes in position or destination.

5.1.5 Behavior Management Subsystem

The behavior management subsystem (*behavior_pkg*) is implemented through the *behavior_manager_node*, which acts as the central decision-making component of the system. This node subscribes to multiple inputs including */vpfs/current_fare*, */odometry/filtered*, and */ultrasonic/detected*, allowing it to determine the robot's current state and operating conditions. Based on this information, it publishes */autonomy/driving_allowed* to control whether the robot should be actively moving, as well as */navigation/current_pose* and */navigation/target_pose* for use by the navigation subsystem.

By combining perception, localization, and safety inputs, this subsystem enables high-level coordination of robot behavior, including task execution such as navigating to pickup and drop-off locations while ensuring safe operation. This node was coded in C++ to improve execution efficiency and reduce latency when processing multiple real-time inputs.

5.2 Integration Strategy, Challenges, and Resolutions

System integration happened over stages rather than all at once. Since the project combined embedded hardware, machine learning inference, route planning, and real time control, the team validated subsystems independently before integration into the full autonomy system.

The first stage of integration focused on hardware and low-level validation. The Raspberry Pi, camera, coral TPU, ultrasonic sensor, LEDs, and supporting hardware were mounted and tested in the PiCar. Once all motors and servos could be activated, and the camera, coral TPU, and Ultrasonic sensor were detected and publishing data, the team moved to software level integration. This stage also included verifying emergency stop and other basic safety-related hardware could be triggered reliably before more advanced testing started.

The second stage integrated the perception and lower-level driving controls. Camera frames were fed into the segmentation pipeline, which generated a lane mask for the autonomy controller. The controller then converted this to steering and speed commands which are passed to the hardware control. This stage involved a lot of iterative testing for both the semantic segmentation model, and the PD controller. If a turn was missed, parameters such as the P and D constants for the steering controller were iteratively adjusted based on how the robot failed. If the tuning wasn't the issue, test footage of the generated mask was reviewed to ensure it was accurate. Due to changing environment conditions like time of day, new buildings in Quackston, and a bright screen beside the town, multiple versions of the model were produced, each with more training data than the last. A similar approach was used to test the tank steering at intersections. Once this low-level driving loop was completed, the taxi and VPFS functionality could be integrated.

The third stage added navigation and the taxi fare functionality. VPFS data was integrated with the mapping system and planner to generate routes and turn sequences. These sequences could then be linked with the lower-level driving controls to execute the route. Once it was manually verified that the generated routes were valid, a debug overlay was created to assist in validating the robot's behaviour. It included its current behaviour state, the segmentation mask, the turn queue, and other line following parameters. This approach was very iterative and allowed the system to be reviewed when unexpected behaviour occurred.

A visual can be seen in figure 6 of Appendix C.

5.3 Implementation Artifacts and Validation

The system was developed and maintained using a well-organized set of implementation artifacts for reproducibility and ease of collaboration. The project is hosted entirely on GitHub as a repository. It contains detailed logbook entries of development as well as the ROS2 workspace containing the packages for each subsystem, including *hardware_pkg*, *behavior_pkg, navigation_pkg*, and *perception* package.

Each package contains source code required to run its respective subsystem. Key nodes such as *cmd_vel_to_robocar*, *behavior_manager_node*, *map_planner*, and *vpfs_queue_bridge* are implemented within these packages, with clear separation of responsibilities. Configuration files, such as map definitions for the map planner and parameter settings, are also included to support navigation and system tuning. In the scripts folder in the ROS2 workspace, there are various launch files used for launching individual packages as well as the entire system in unison.

To ensure consistency across development environments, Docker containers were used to provide a consistent setup for the team. This eliminated dependency issues and allowed the system to be deployed and tested reliably across multiple machines.

Individual nodes were tested independently using ROS2 command-line tools such as *ros2 topic pub* and *ros2 topic echo*. For example, the *cmd_vel_to_robocar* node was validated by manually publishing velocity commands to */cmd_vel* and observing the car's hardware responses. Similarly, the *ultrasonic_node* was tested by monitoring */ultrasonic/detected* to test accurate obstacle detection. This stage was beneficial in testing each subsystem in isolation before integration.

Following unit testing, subsystems were incrementally integrated to verify communication between nodes. For instance, the perception subsystem was tested by connecting the segmentation node to the autonomy node to ensure that a camera feed was visible. The behavior manager was then introduced to incorporate decision-making logic with the ultrasonic sensor, followed by integration with the navigation subsystem for path planning. This staged integration process reduced debugging complexity by allowing faults to be traced back to specific subsystem interactions.

Once all subsystems were integrated, full system testing was conducted on the PiCar-X. These tests evaluated the robot's ability to perform complete tasks, including lane following, intersection handling, obstacle avoidance, and navigation to pickup and drop-off locations using VPFS. Successful demonstrations of these tasks verified that the system operated properly in the real-world environment.

5.4 Development Timeline and Milestone Review

Despite the structured progression for full system implementation, integration was deferred until the final week of the project. While this approach allowed for rapid progress on individual components, it resulted in a compressed integration timeline. A significant amount of debugging and system-level validation had to be performed within a short period as a result. The development plan was very strong but did not account for all the testing.

The integration phase showed some complexities that were not obvious during isolated subsystem testing. Debugging the full system proved to be challenging due to the various interacting ROS2 nodes and topics. Issues such as inconsistent message timing, unexpected interactions between subsystems, and difficulty tracing errors across multiple nodes increased the overall complexity of debugging. This required quick troubleshooting given the time constraint the team was put into.

Due to the limited time available for full system validation, some issues were apparent in the final competition. These included confusing errors in system behavior and reduced reliability. While the core functionality of the system was operational, the poorly planned integration phase impacted overall robustness during the competition.

Despite these challenges, the project successfully demonstrated integration of multiple complex subsystems into a cohesive autonomous system. The experience highlighted the importance of earlier system-level integration and continuous testing throughout development. This experience shows the importance of integrating subsystems earlier in the development process. While modular design enabled efficient independent development, delaying integration introduced significant risk and complexity. Future projects would benefit from iterative integration strategy that enforces progressive development and combining of subsystems. And of course, allocating more time for system-level debugging and validation would improve overall reliability and performance.

6. Verification, Validation, and Performance Evaluation

6.1 Performance Metrics

The semantic segmentation model was first evaluated during training and validation in Google Colab. The best model achieved a validation accuracy of approximately 98.9% with a validation loss of 0.0323, above the teams accuracy goal of 96%, indicating strong pixel-wise classification performance on the validation dataset. The trained model was then tested on recorded driving footage and live vehicle trials to confirm that it could reliably distinguish background, green lane markings, and yellow stop lines under realistic operating conditions. This provided quantitative evidence that the perception pipeline was suitable for downstream control and stop-line detection.

Lane-following performance was evaluated through repeated driving trials on straight segments, left turns, and right turns within the Quackston track. The vehicle successfully completed all consecutive straight-line trials on the Quackston map, 4/4 left-turn trials, and 3/4 right-turn trials without unsafe lane departure. The reduced success rate for right turns was likely caused by poorer lighting in that section of the track, which affected the consistency of lane perception. Even with this limitation, the system showed stable steering with minimal oscillation and remained within the intended drivable path in nearly all runs. The team believes that additional data collection and retraining under those lighting conditions would likely improve performance further. These results were used to assess the combined performance of the perception model and PD steering controller.

Stopping performance was measured by testing the vehicle's response to stop conditions, including stop lines and required stopping locations. In 20 trials, the vehicle came to a complete stop in all cases. Obstacle-response performance was measured separately using controlled ultrasonic sensor tests, where an obstacle was introduced within the stopping threshold. The emergency stop system successfully halted the vehicle in 5/5 trials. These tests were used to verify that the vehicle could respond safely to both planned and unplanned stopping conditions.

End-to-end system reliability was evaluated using full autonomous fare-style runs that combined perception, control, signalling, tracking, and passenger transport into one integrated test. The vehicle successfully completed 2/4 full-system trials, corresponding to an overall completion rate of 50%.

6.2 Qualitative Observations

Qualitative testing of the vehicle was overwhelmingly positive. The system demonstrated exceptionally strong driving performance, with speed and lane-following behaviour that stood out clearly relative to other teams observed during testing. The vehicle was consistently able to move quickly while maintaining stable control, smooth steering corrections, and strong line-tracking performance on both straight segments and turns. Its compact physical design also provided a clear advantage during tank turns, allowing it to rotate and reorient faster than other observed vehicles. This contributed to particularly strong maneuverability at intersections and during sharp directional changes.

The team's line-following and turning performance were among the strongest observed by the team during testing. Straight-line performance was especially consistent, and left turns were completed very reliably. Right turns were also generally successful, with only occasional issues under poorer lighting conditions in one section of the track. Even in these cases, the vehicle usually recovered well and maintained competitive performance. Overall, the perception and control system gave the vehicle a level of speed and responsiveness that made it noticeably stronger than many other cars on the track.

This strong performance was visible enough that other teams approached the group for advice and assistance after seeing how quickly and effectively the car was operating. In addition to its speed, the

system also showed strong safety-related behaviour. The duck passenger remained securely restrained throughout testing, with no tipping or sliding. Stop detection was also generally effective, although the vehicle occasionally produced false positive stops. This was considered an acceptable and even preferable trade-off, since the team intentionally prioritized avoiding missed stops over eliminating every unnecessary stop.

The main remaining limitation was in obstacle detection coverage rather than general vehicle control. Due to the fact that the ultrasonic sensor was forward-facing, the system was most effective when hazards were directly in front of the vehicle and somewhat less robust for objects outside that sensing range. Despite this, the overall qualitative assessment of the system was highly positive. The vehicle combined excellent speed, strong turning performance, reliable lane following, and secure passenger handling, making it one of the stronger-performing systems observed in the competition setting.

6.3 Comparison to Objectives

The final system met most of the project's main objectives. The strongest alignment was in safe autonomous driving behaviour, where the vehicle demonstrated reliable lane following, consistent stopping, secure duck transportation, and functioning safety and signalling systems. The perception model also exceeded the team's target validation accuracy, supporting the objective of using vision-based perception for control and stop-line detection, as opposed to the grayscale sensor.

The vehicle also performed strongly with respect to speed, maneuverability, and overall driving quality. Straight-line performance and left turns were completed very reliably, and right turns were mostly successful, showing that the system was capable of handling the main track features while maintaining stable control. The CARE system also fully met its objective, as the duck remained secure throughout testing, and the stopping system met expectations in both stop-condition and controlled obstacle-response tests.

The objective of achieving fully autonomous taxi operation was met to a meaningful degree. The vehicle successfully completed 2 of 4 full trials, which represented a strong result relative to the observed performance of many other teams. These results showed that the system was capable of integrating perception, control, signalling, tracking, and passenger transport into a functioning autonomous taxi platform. Although reliability under full integration was not yet perfect, the system demonstrated that the core objective had been achieved in a competitive and credible way. Overall, the final results aligned well with the team's initial goals and showed that most core objectives were successfully achieved.

6.4 Competition Results

The final competition results were not used as the basis for evaluating system performance. During the competition, the vehicle did not operate as intended due to a software version control issue rather than a fundamental failure of the design itself. Code had been pushed to GitHub from the wrong directory, and the local codebase subsequently reverted to an older version, which prevented the most recent and best-performing implementation from being run during the event.

As a result, the behaviour observed during the competition was not representative of the system's actual validated capabilities. For this reason, controlled testing and recorded trial results were used instead as the main basis for performance evaluation. These tests more accurately reflected the final integrated system, including the perception model, lane following, stopping behaviour, and autonomous fare-style operation.

Although the competition run itself was unsuccessful, the team does not consider it an accurate measure of the vehicle's performance. The issue was primarily procedural and related to software deployment, rather than the underlying hardware, logic, or code. Based on the results obtained during

structured testing, the team believes the system would have performed competitively in the competition environment had the correct final code version been deployed.

6.5 Supporting Materials

Figure 3: Model Accuracy compared to target accuracy of 96%

7. Limitations, Risks, and Recommendations

7.1 Challenges and Limitations of the Final System

One major challenge in this project came from using Ubuntu and ROS rather than a simpler OS. While this decision provided valuable experience with a more advanced robotics software stack, it also introduced additional setup complexity, including driver installation, hardware interfacing, and system configuration. Considerable effort was required to ensure that the Raspberry Pi, Robot HAT, camera, Coral Edge TPU, and other hardware components could all operate within the Ubuntu environment.

Another potential limitation of the final system was the decision to rely entirely on the camera for line following rather than implementing the grayscale sensor as a fallback. In the final system, the camera worked well and provided much richer perception, since it allowed the vehicle to see further ahead rather than only directly beneath the chassis. Ideally, the team would have also integrated the grayscale sensor as a secondary fallback layer to improve robustness in cases where vision performance degraded. However, this was not a major focus within the project scope, and because the camera performed strongly overall, the additional complexity of implementing a full fallback architecture was not prioritized. As a result, the system achieved strong performance using vision alone, but it remained more dependent on camera input than it ideally would have been in a more fully developed design or real world autonomous vehicle.

7.2 Assumptions

The team made several important assumptions that influenced the design of the system. One major assumption was that the competition environment would be controlled and predictable enough for a camera based perception system to work reliably on its own. Unlike real autonomous vehicles, the project did not need to account for conditions such as rain, snow, or fog that could obscure the camera. This made it reasonable to rely solely on vision for line following and stop line detection.

Another key assumption was that the team would know the structure of the track in advance, including the appearance of the lanes, intersections, and other major road features. Because the lane markings and environment were known beforehand, the perception system could be trained specifically for that setting rather than needing to generalize to a wide range of unknown road conditions. This assumption significantly shaped the decision to use a semantic segmentation model trained specifically for Quackston.

These assumptions largely held during testing and allowed the team to build a system that performed strongly within the intended competition environment. However, they also limited the generality of the design. The vehicle was effective in a controlled setting, but it was not intended to handle the broader uncertainty and environmental variation that would be expected in a real-world autonomous driving system.

7.3 Lessons Learned

This project provided valuable experience in both technical development and project execution. One of the most important lessons was the benefit of working with ROS and Ubuntu in a full embedded robotics system. Although this introduced additional difficulties, it gave the team practical experience with modular node-based software design, inter-node communication, and hardware-software integration in a way that was much closer to robotics development than a simpler script-based approach. The project also reinforced how important structured system integration is, since bringing together perception, control, signalling, localization, and hardware actuation required much more coordination than developing any one subsystem individually.

Another major lesson was the importance of testing as early and as often as possible. Some parts of the project, particularly VPFS integration and full testing on the Quackston mats, could only be exercised properly near the end of the project timeline. Because access to the full testing environment was limited, the team had fewer opportunities than ideal to iterate on complete end-to-end behaviour under competition conditions. This highlighted the importance of validating key external dependencies and final operating conditions early in the design cycle, since earlier access would have allowed more refinement, more robust tuning, and better preparation for competition conditions.

The project also showed that successful autonomous system design depends on more than just making each subsystem work well on its own. Even when individual components such as perception, stopping, and line following performed strongly, full-system reliability still depended on how well those parts interacted. Overall, the team learned that strong results come not only from good algorithms and hardware choices, but also from early testing, repeated iteration, and careful integration management.

7.4 Societal and Environmental Impacts

Autonomous vehicles can provide important social benefits, including improved mobility, reduced human driving error, and greater accessibility for people who cannot drive independently. A system like this could help enable cheap automated transport in controlled environments while also encouraging more consistent rule-following and safety-focused operation. However, autonomous vehicles also raise concerns around public trust and safety, meaning their benefits depend on reliability and responsible deployment.

From an environmental perspective, autonomous vehicles may reduce inefficient driving, unnecessary braking, and idling, which could improve energy efficiency. At the same time, these systems require added computing hardware, sensors, and electrical power, so the overall environmental benefit depends on how efficiently the platform operates. In this project specifically, the design was compact, lightweight, and safety oriented, which would be beneficial if scaled to structured environments.

However, the system also depended on a controlled setting and known road features, so its current form would not yet be suitable for broader real-world use. Overall, the project suggests that autonomous transport can provide meaningful societal and environmental benefits, but only when safety, reliability, and efficiency are prioritized together.

7.5 Recommendations for Future Work

Future iterations of the system should focus primarily on improving robustness and full-system reliability. One important recommendation is to collect additional training data under a wider range of lighting conditions and retrain the semantic segmentation model to improve consistency in less ideal visual environments. A second recommendation is to integrate the grayscale sensor as a fallback, which would reduce the system's dependence on camera input alone.

The team would also recommend expanding obstacle detection coverage, since the forward-facing ultrasonic sensor was effective only for hazards directly in front of the vehicle. In addition, future work should place greater emphasis on earlier end-to-end testing. Finally, the project highlighted the importance of finishing the system earlier so that more time can be dedicated to end-to-end testing and refinement. Having a longer period for final validation would reduce the risk of deployment mishaps, improve confidence in the final implementation, and allow the team to enter competition runs with a more stable system.

8. Conclusions

8.1 Summary and Closing Remarks

This project simulated the problem of developing an autonomous taxi capable of operating in Quackston while meeting various safety requirements. To solve this problem, the team designed and integrated a system built around a semantic segmentation model for perception, PD controller for steering, ultrasonic sensor for obstacle detection, signalling, fare interaction, and secure passenger transport. The methodology combined model training and validation, subsystem integration in ROS, and repeated testing of lane following, stopping, turning, and fare operation.

The final results showed that the system performed strongly in several key areas. The perception model exceeded the team's target validation accuracy, lane following and turning were highly competitive, stopping behaviour was reliable, and the duck passenger remained securely restrained throughout testing. Although the final competition performance was affected by a deployment issue, controlled testing demonstrated that the vehicle was capable of autonomous behaviour. Overall, the project was significant not only because it produced a fast and functional autonomous taxi platform, but also because it gave the team valuable experience in robotics, perception, embedded systems, and full-system development.

8.2 Future Work

Future iterations of the course project would benefit from moving the pre competition validation earlier in the schedule, at least one week before the final competition. Requiring teams to reach a baseline earlier would encourage more disciplined progress throughout the term and reduce the risk of major last-minute issues affecting competition performance.

An earlier pre-check would also make the final weekend more productive. Rather than still troubleshooting core autonomy or hardware integration, teams could use that time mainly for final VPFS integration, tuning, and full autonomy verification. This would better match the way the system is actually developed, since many core behaviours such as line following, stopping, turning, and signalling can be tested without needing the full Quackston mats to be set up. Earlier validation would therefore leave more time for refinement, reduce pressure at the end of the project, and improve the overall quality and reliability of the final competition systems.

Another area for future improvement is the competition evaluation structure. In practice, the grading did not always appear to reward the strongest overall systems relative to their demonstrated technical performance. A more performance-based scoring structure could better reflect actual vehicle capability. For example, fare completion could be graded in tiers, such as partial credit for completing one fare and increasing credit for completing additional fares, rather than creating a larger gap between teams whose practical performance may have been more similar than the final outcome suggested.

The relationship between money, safety, and penalties could also be reconsidered. At present, traffic violations had such a strong negative effect that it was not really feasible for teams to intentionally trade safety for money, even though the project framing suggested that this type of ethical trade-off was part of the design challenge. A better structure might separate financial reward from safety penalties more clearly, so that teams could make more meaningful design decisions about whether to prioritize speed and money or safer, more conservative behaviour. This would create a more realistic ethical design trade-off and better reflect the course objective of balancing performance, safety, and responsibility.

References

[1] SunFounder, "Robot HAT," SunFounder Documentation. [Online]. Available: https://docs.sunfounder.com/projects/picar-x/en/latest/hardware/robot_hat.html. [Accessed: Jan. 20, 2026].

[2] SunFounder, "About PiCar-X: AI-Powered Autonomous Robot," SunFounder Products. [Online]. Available: https://www.sunfounder.com/products/picar-x. [Accessed: Jan. 20, 2026].

[3] Google Coral, "USB Accelerator Datasheet (v1.4)," Coral.ai. [Online]. Available: https://coral.ai/docs/accelerator/datasheet/. [Accessed: Jan. 20, 2026].

[4] Google Coral, "Edge TPU Performance Benchmarks," Coral.ai. [Online]. Available: https://coral.ai/docs/edgetpu/benchmarks/. [Accessed: Jan. 20, 2026].

[5] Open Robotics, "ROS 2 Documentation: Foxy Fitzroy," ROS.org. [Online]. Available: https://docs.ros.org/en/foxy/. [Accessed: Jan. 22, 2026].

[6] Gazebo, "Gazebo: Robot Simulation Made Easy," Gazebosim.org. [Online]. Available: https://gazebosim.org/home. [Accessed: Jan. 21, 2026].

[7] Raspberry Pi Foundation, "Pi 5 Power Consumption and Undervoltage Benchmarks," Raspberry Pi Forums. [Online]. Available: https://forums.raspberrypi.com/. [Accessed: Jan. 21, 2026].

[8] SunFounder, "Hardware: Camera, Ultrasonic, and 3-pin Battery Modules," SunFounder Documentation. [Online]. Available: https://docs.sunfounder.com/projects/picar-x/en/latest/hardware/modules.html. [Accessed: Jan. 21, 2026].

[9] D. Minott, S. Siddiqui, and R. J. Haddad, "Benchmarking Edge AI Platforms: Performance Analysis of NVIDIA Jetson and Raspberry Pi 5 with Coral TPU," Georgia Southern University, Statesboro, GA, USA, 2024.

[10] Google Coral, "Edge TPU Compiler and Workflow Documentation," Coral.ai. [Online]. Available: https://coral.ai/docs/edgetpu/compiler/. [Accessed: Jan. 21, 2026].

[11] K. Aziev, "Picar-X Racer: Advanced AI Vision and Autonomous Navigation," GitHub Repository. [Online]. Available: https://github.com/kaziev/picar-x-racer. [Accessed: Jan. 21, 2026].

[12] J. Geerling and G. Halfacree, "Raspberry Pi 5 talking to a Google Coral TPU," Jeff Geerling, 2024. [Online]. Available: https://www.jeffgeerling.com.

[13] Google Coral, "Get Started with the USB Accelerator," Coral.ai. [Online]. Available: https://coral.ai/docs/accelerator/get-started/. [Accessed: Jan. 22, 2026].

[14] Daniel, "Setup Coral TPU USB Accelerator on Raspberry Pi 5 with Docker," DIYEngineers, 2024. [Online]. Available: https://diyengineers.com.

[15] "Lane Line Detection based on Machine Vision," in 2022 4th Int. Conf. on Power, Intelligent Computing and Systems (ICPICS), Shenyang, China, 2022.

[16] S. Smith, "Programming the SunFounder PiCar-X," Stephen Smith's Blog. [Online]. Available: https://stephensmith.blog/. [Accessed: Jan. 21, 2026].

[17] V. Jyothi et al., "Miniature Autonomous Vehicle Development on Raspberry Pi," in 2018 IEEE 14th Int. Conf. on Intelligent Computer Communication and Processing (ICCP), Cluj-Napoca, Romania, 2018.

[18] SunFounder Community, "PiCrawler and PiCarX Battery Supply Issues with Raspberry Pi 5," SunFounder Forums. [Online]. Available: https://forum.sunfounder.com/. [Accessed: Jan. 20, 2026].

[19] J. Frazelle, "Power Consumption Benchmarks for Raspberry Pi Models," Pi Dramble Wiki. [Online]. Available: https://pidramble.com/wiki/benchmarks/power-consumption. [Accessed: Jan. 21, 2026].

[20] H. Maghsoumi and Y. Fallah, "A Small-Scale Robot for Autonomous Driving: Design, Challenges, and Best Practices," arXiv preprint arXiv:2506.15870, 2025.

[21] S. Tra, "Autonomous Driving: Modular Pipeline vs End-to-End and LLMs," Medium, 2024.

[22] J. Chen, X. Huang, and K. Huang, "A Deep Reinforcement Learning-Based Autonomous Vehicle Control for Unstructured Environment," IEEE Access, vol. 12, pp. 64283–64303, 2024.

[23] T. K. Nguyen et al., "Analysis of Object Detection Models on Duckietown Robot Based on YOLOv5 Architectures," Int. J. of iRobotics, vol. 4, no. 4, pp. 17–22, 2021.

[24] M. Sukkar, R. Jadeja, M. Shukla, and R. Mahadeva, "A Survey of Deep Learning Approaches for Pedestrian Detection in Autonomous Systems," IEEE Access, vol. 13, 2025.

[25] "Lane Line Detection based on Machine Vision," in 2022 IEEE 4th Int. Conf. on Power, Intelligent Computing and Systems (ICPICS), Shenyang, China, 2022.

[26] S. Pareigis, D. L. Riege, and T. Tiedemann, "Miniature Autonomous Vehicle Environment for Sim-to-Real Transfer in Reinforcement Learning," ScitePress, 2025.

[27] H. Jain and P. Babel, "A Comprehensive Survey of PID and Pure Pursuit Control Algorithms," arXiv preprint arXiv:2409.09848, 2024.

[28] "Control strategies for autonomous vehicle path tracking: A comparative study of PID, Pure-Pursuit, and Stanley methods," ResearchGate, 2024.

[29] Google Coral, "USB Accelerator Datasheet (v1.4)," Coral.ai, Aug. 2020. [Online]. Available: https://coral.ai/docs/accelerator/datasheet/.

[30] Google Coral, "Edge TPU Compiler and Workflow Documentation," Coral.ai, 2022. [Online]. Available: https://coral.ai/docs/edgetpu/compiler/.

[31] Cyberbotics, "Webots: Open Source Robot Simulator," Cyberbotics.com, 2025. [Online]. Available: https://cyberbotics.com.

[32] K. S. Lanchukovskaya, D. E. Shabalina, and T. V. Liakh, "Semantic Image Segmentation Methods in the Duckietown Project," Duckietown, 2024.

[33] M. Codevilla et al., "Exploring the Limitations of Behavior Cloning for Autonomous Driving," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), Seoul, Korea, 2019.

[34] M. Bojarski et al., "End-to-End Learning for Self-Driving Cars," arXiv preprint arXiv:1604.07316, 2016.

[35] C. Coutinho, "Road Lane Detection for Autonomous Robot Guidance," in IEEE Int. Conf. on Autonomous Robot Systems and Competitions, Espinho, Portugal, 2014.

[36] "Small Object Detection in Traffic Scenes: Challenges and Solutions," MDPI Electronics, vol. 14, no. 13, 2025.

[37] D. Li, P. Auerbach, and O. Okhrin, "Towards Autonomous Driving with Small-Scale Cars: A Survey of Recent Development," Duckietown, 2024.

[38] ELEC 392 Course Staff, "Welcome to the Town of Quackston," ELEC 392 GitBook, Jan. 2026. [Online]. Available: https://elec392.gitbook.io/elec-392/8Ty9Nx3l84dHok0cVgCY/competition-information/welcome-to-the-town-of-quackston. [Accessed: Jan. 25, 2026].

[39] Mattel, "Disney Pixar Cars Global Racers Cup Lightning McQueen," Amazon.ca, 2025. [Online]. Available: https://www.amazon.ca/Vehicles-Lightning-Collectible-Character-5-5-Inch/dp/B09NYPDJJD. [Accessed: Jan. 26, 2026].

[40] Queen's University, Academic Integrity, 2025. [Online]. Available: https://www.queensu.ca/academicintegrity/. [Accessed: Jan. 25, 2026].

Appendix A: Team Collaboration and Professional Practice

A.1 Roles and Responsibilities

Refer to the Work Breakdown table on page i for the specific report sections contributed by each member. The team assigned the following engineering roles:

Team Member	Primary Responsibility	Secondary Responsibility
Ben Malvern	Integration	Hardware Setup/Debugging
Clarke Needles	Autonomy	Perception Decisions
Filip Radenovic	Hardware	Safety/Data Collection
Jimmy Moutafis-Tymcio	Computer Vision/Perception	Steering Logic (Line Following, turning)

A.2 Expectations for Participation and Initiative

Every member was expected to take initiative in completing their work. If a member was stuck or unsure, they communicated it immediately, asking for help early was treated as responsible behaviour.

- Extenuating Circumstances: Life happens. If an emergency or illness impacted a member's work, they notified the team as soon as possible so the load could be redistributed.

A.3 Decision-Making and Conflict Resolution

The team made technical decisions in accordance with the Team Operating Agreement. Routine technical choices were made by the subsystem lead based on research and implementation testing. For major system architectural decisions, the team planned a design meeting to generate ideas and select one approach based on feasibility and project constraints.

The team strived for consensus. If agreement could not be reached after reasonable discussion, a majority vote was taken. Disagreements were strictly technical, no individual favoritism for a given solution. In the case of no clear majority, intricate technical disputes were deferred to team members with relevant experience.

A.4 Workload Equity

The team ensured workload equity through: overlapping responsibilities (each subsystem had a secondary owner who could maintain progress despite delays); allocating tasks based on milestones (work distributed evenly across project phases); and sharing documentation (all documents and code repositories were accessible to all team members, making contributions fully transparent).

A.5 What Worked Well and What Required Adjustment

What worked well was the team's division of responsibilities across major subsystems, which allowed members to develop strong ownership while still supporting one another when needed. Communication was generally effective, and team members were willing to assist outside their primary roles when integration challenges arose. What required adjustment was the timing of full-system integration and testing, since some critical components could only be validated late in the project. This showed that while the collaboration structure was strong, earlier end-to-end testing would have improved coordination and reduced last-minute pressure.

Appendix B: Individual Reflection

B.1 Ben Malvern

My contributions to the project were mainly in system integration, navigation, autonomy logic, testing, and low-level hardware setup. I played a major role in getting the platform running at a practical level including setting up Ubuntu and ROS 2 on the Raspberry Pi, getting the camera and Robot Hat working on an unsupported OS, and integrating motor and servo control into ROS. In addition to this, I was responsible for the implementation of the graph-based mapping algorithm, used by the planner, and integrating the route planning with the autonomy system to execute turns. My most significant contribution was the system integration and debugging work I did, which included creating testing scripts to significantly speed up testing time, and making debug overlays to monitor ROS topics during operation for replay.

One of the biggest areas of growth for me in this project was in testing, debugging and systems-level thinking. I learned a lot about how perception, control, navigation, and hardware interfacing all depend on each other. I also learned how when facing problems in autonomous systems, they often come from interactions between multiple subsystems, so even if they work individually, they may not work together. This project gave me much stronger practical experience in robotics integration and helped me understand how the different parts of a system this complex all can come together.

A gap I identified in my own practice is that I can sometimes become too focused on one suspected cause when debugging, which can make it harder to step back and evaluate the full system. I also learned an important lesson about version control late in the project, when git mistake caused recent changes to be lost at a critical time. That experience showed me how important careful source control practices, backups, and communication are in a collaborative engineering environment. In future work, I would be much more deliberate about validating branches, protecting major changes, and coordinating final integrations more carefully. Overall, the project strengthened both by technical skills, and my ability to work through complex integrated problems.

B.2 Clarke Needles

My contributions to the project were primarily in system architecture, behavior management, localization (VPFS integration), and hardware interfacing. I was responsible for designing the initial system architecture diagram, which served as a template for how the system should look and behave, as well as how subsystems communicate through ROS2 topics. While the final implementation changed slightly from the original design, the structure remained consistent and helped guide the overall development process. I also implemented the VPFS integration originally through the vpfs and odometry nodes, but later these were combined into the vpfs_queue_bridge. This subsystem allowed the robot to interface with the VPFS which was beneficial for receiving global position and task data. I also helped to develop the behavior_manager_node, which coordinated the decision-making across perception, navigation, and safety inputs. I also worked on the ultrasonic node and contributed to the system-level integration, ensuring that sensor data could be reliably used for real-time decision-making.

One of the biggest areas of growth for me was in system-level thinking and integration. I gained a much deeper understanding of how multiple subsystems, such as perception, navigation, and hardware control must work together in a real-time system. I learned that even when individual components function correctly, unexpected issues often arise during integration due to timing, communication, or interaction effects. A gap I identified in my approach is that integration and system-level validation were left too late in the development process, which resulted in increased complexity and time pressure

during final debugging. This experience reinforced the importance of earlier and more continuous integration, as well as structured testing throughout development. Overall, this project strengthened both my technical ability to design and implement complex robotic systems and my understanding of how to manage and coordinate large-scale system integration.

B.3 Filip Radenovic

My contributions to the project were primarily in hardware support, data collection, and the documentation of our system's ethical and social impact. I focused on the physical assembly and maintenance of the PiCar-X platform, ensuring that the hardware remained stable for testing. I was responsible for the CARE (Cabin and Restraint Equipment) system, which utilized a simple but effective cardboard and duct tape design to keep our duck passengers secure during maneuvers.

This project was a major learning opportunity for me in areas where I had little prior experience, specifically ROS 2, Computer Vision (CV), and Machine Learning (ML). By assisting with the data pipeline, I learned how a MobileNetV2-based model is trained in Google Colab and then quantized for the Coral TPU to achieve high-speed inference. I also developed a foundational understanding of how ROS nodes communicate to manage vehicle behavior. A gap I identified is my current level of technical depth in complex autonomy logic compared to my peers. While I could support the implementation, I recognized that I need to continue developing my software skills to contribute more directly to high-level system architecture in the future.

I learned that being a valuable team player often involves identifying how to best support others to ensure the collective success of the group. Even when I was not the primary architect of the software logic, I found that I could help the team succeed by handling hardware reliability and documentation, which allowed the technical leads to focus on debugging the autonomy stack. I made a conscious effort to be reliable and assist whenever a teammate was overwhelmed, learning that a project's success depends on the interaction of all roles, not just the most technical ones. In future projects, I will continue to focus on being a supportive collaborator who prioritizes the team's goals, while also working to bridge my technical gaps so I can take on more diverse responsibilities.

B.4 Jimmy Moutafis-Tymcio

My contributions to the project were primarily in perception, computer vision, and the autonomy logic. I worked extensively on the semantic segmentation model used for line following and intersection detection, including data collection, labelling, training, evaluation, and deployment to the Coral USB accelerator. I also contributed significantly to the autonomy logic, including stop logic, tank steering behaviour, and PD tuning for lane following. These responsibilities required me to work across both software development and system integration, and they gave me a much stronger understanding of how perception and control must interact in a real autonomous vehicle. One of the things I found most rewarding about this project was that the work felt both difficult and feasible. It was challenging enough to be interesting, but still tangible, meaning that progress could be seen directly in the vehicle's behaviour. I enjoyed working on a problem where improvements to the model or logic could produce visible changes in how the car performed because that made the project feel much more meaningful than a theoretical task.

One of the most important areas of growth for me was in computer vision, ROS, and edge computing. Through this project, I developed much stronger practical skills in training and evaluating a vision model, integrating perception into a ROS node, and tuning control behaviour based on real vehicle performance rather than only theoretical expectations. I also learned more about the challenges of deploying machine learning models to constrained hardware. Working with the Coral USB accelerator helped me

better understand edge computing, model optimization, and the tradeoffs between model complexity, speed, and reliability. In addition, I learned a great deal about debugging integrated robotics systems, where problems are often caused by the interaction between multiple subsystems rather than by a single isolated issue. This taught me the importance of testing changes in the full system and not assuming that a component working in isolation would behave the same way once integrated.

I also learned a lot about teamwork through this project, and enjoyed working in a group on a challenging technical problem. A gap I identified in my own practice is that I can sometimes focus too heavily on controlling all aspects of a group project. In past group settings, I often preferred to manage everything directly to ensure it met my standards. This project helped me improve in that area by forcing me to trust other team members to take on larger roles and responsibilities. It also taught me how to handle conflict and disagreements more professionally. There were times when I felt certain technical issues should be prioritized differently, especially during final integration, while other teammates had different views on what mattered most. Working through those situations taught me that disagreement is a normal part of collaborative engineering and that it needs to be handled through clear communication, listening, and compromise. I learned that it is not enough to believe a technical direction is correct; I also need to explain that reasoning clearly, understand other perspectives, and help the team make decisions based on overall project goals and constraints rather than my own preferences. Overall, this project strengthened both my technical skills and my ability to function effectively within a team.

Appendix C: Figures

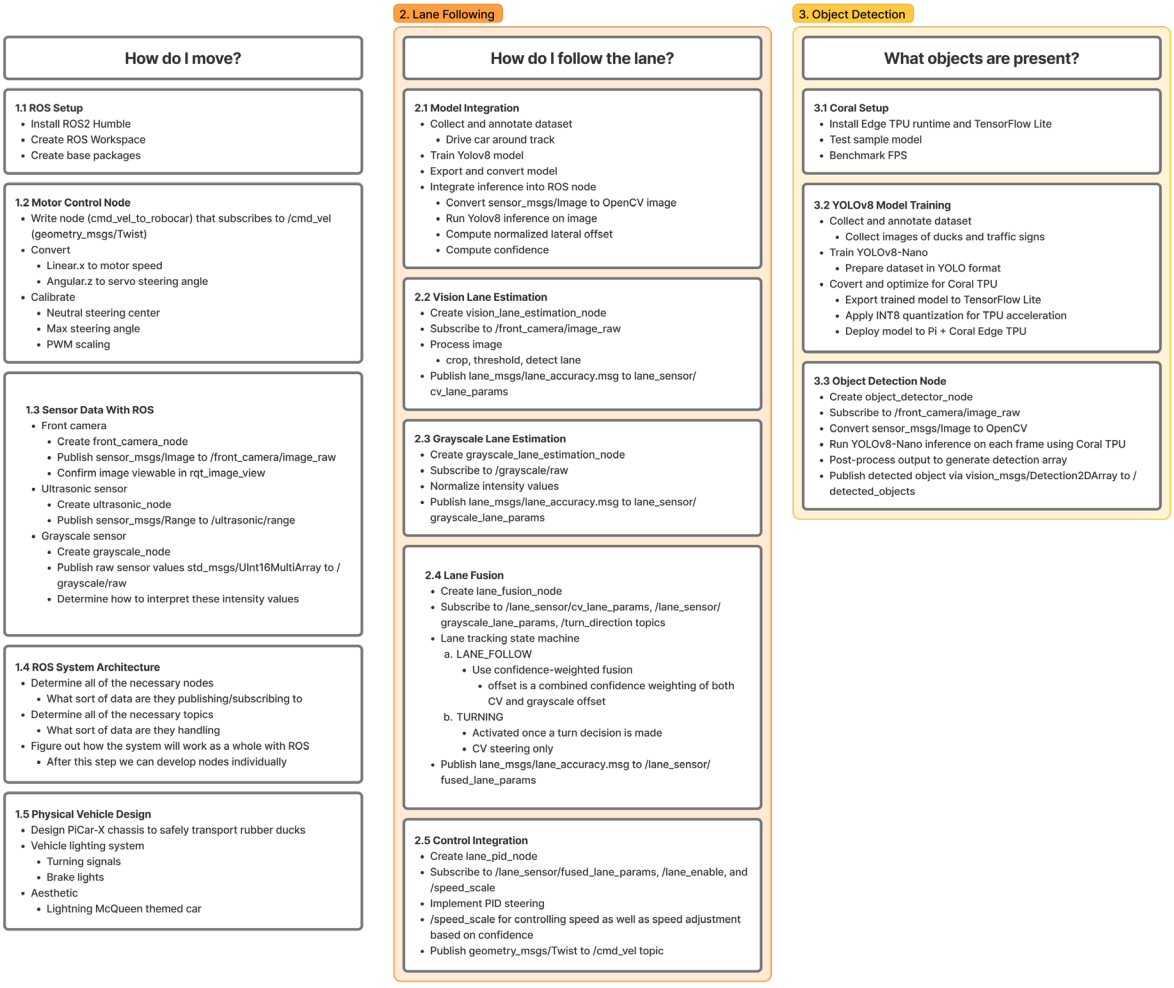

Figure 4: Initial System Development Plan for Motion Control, Lane Following, and Object Detection

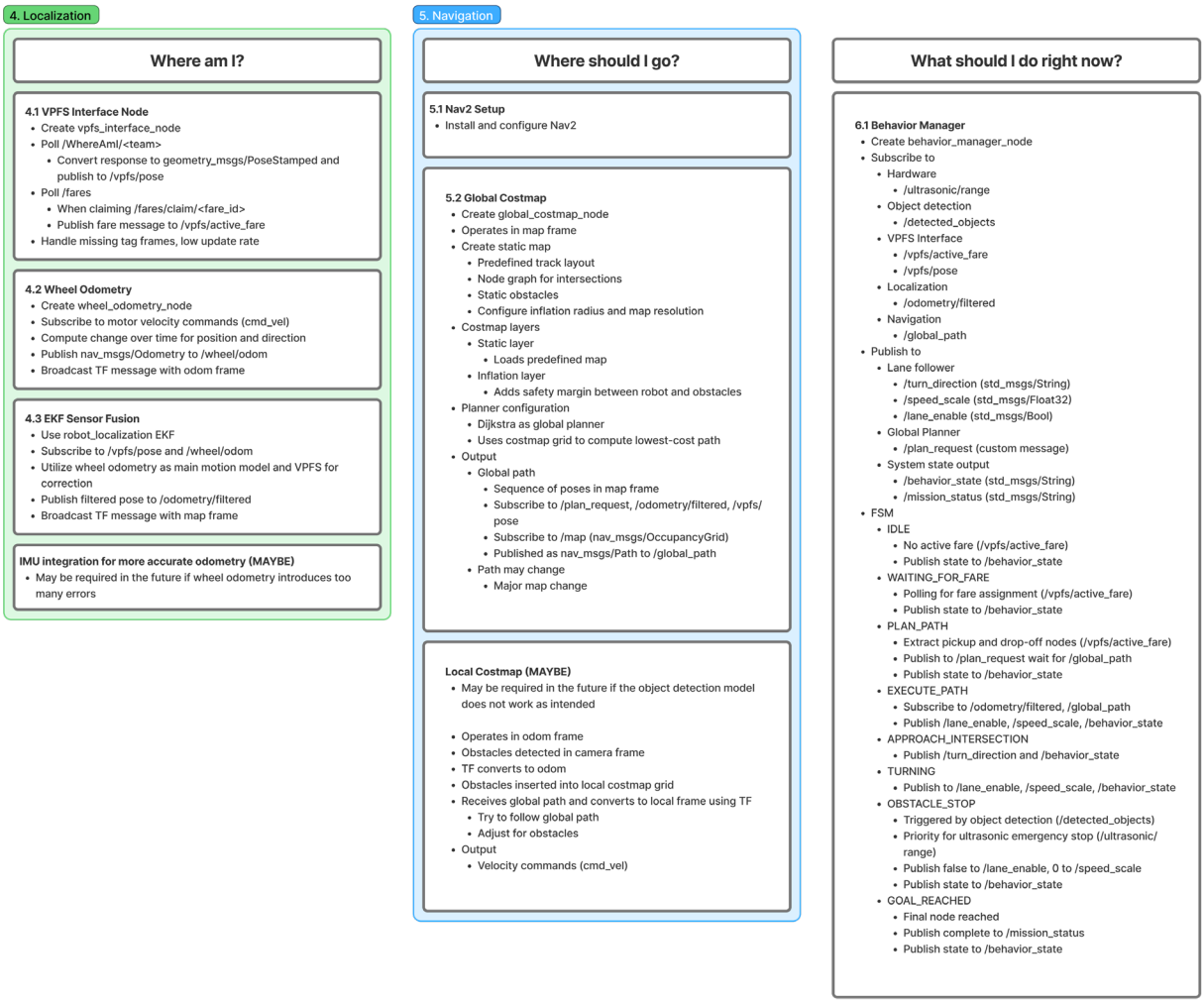

Figure 5: Initial System Development Plan for Localization, Navigation, and Behavior Management

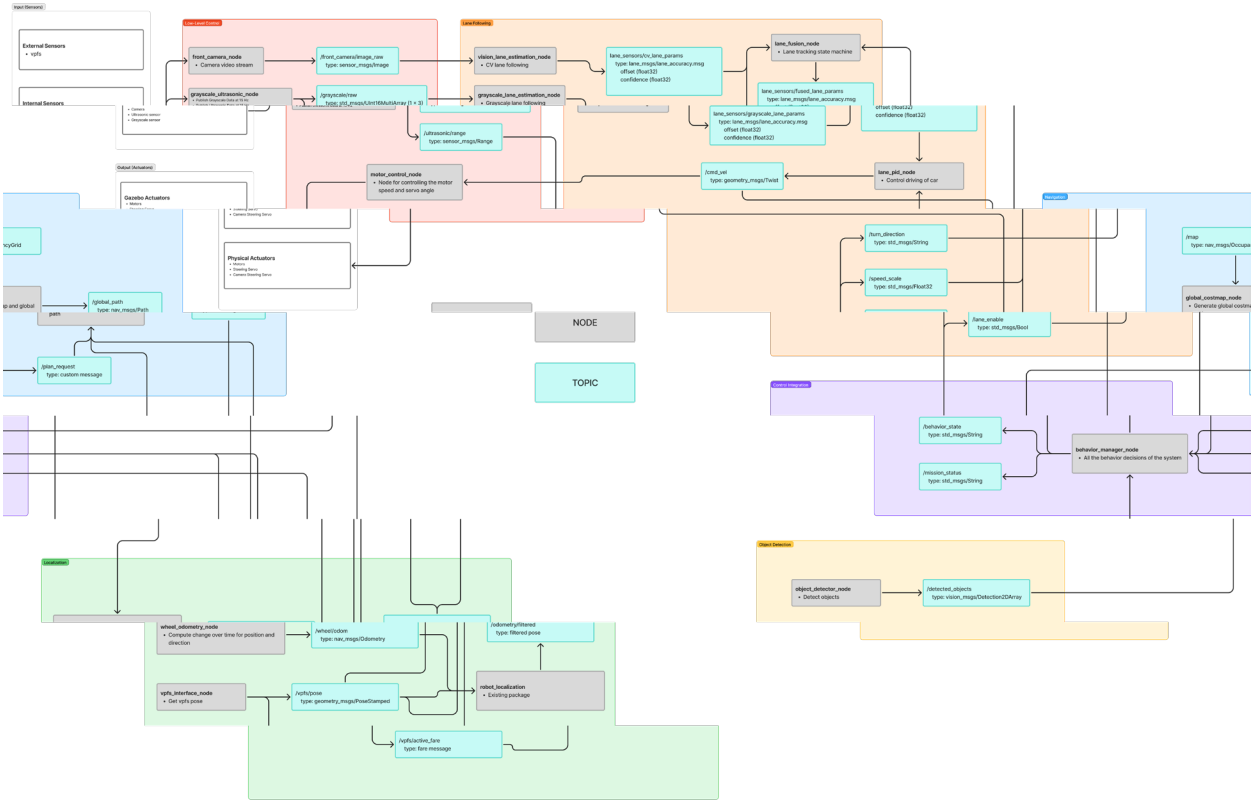

Figure 6: Subsystem Architecture

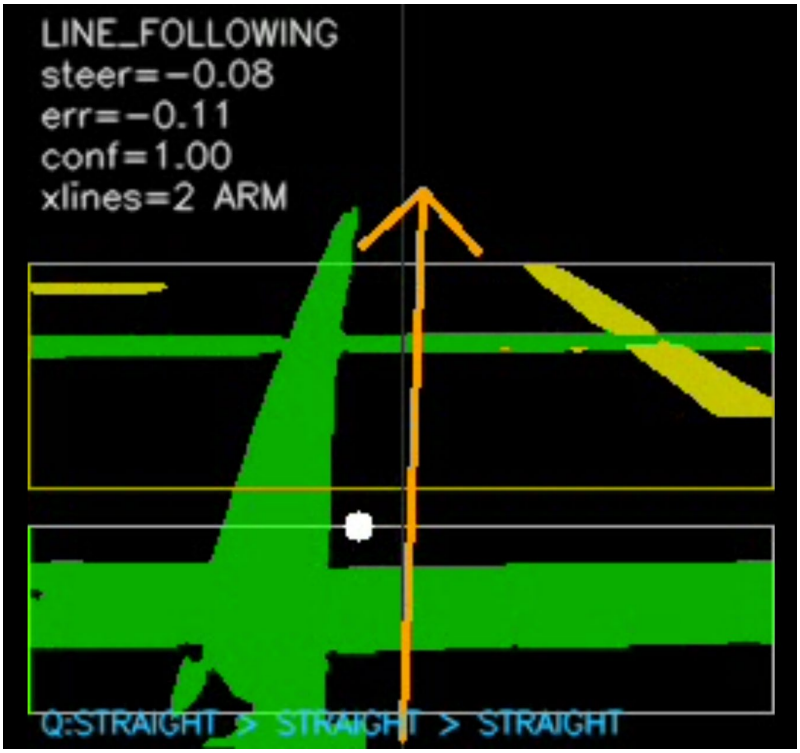

Figure 7: Lane Following Debug Window